

Mappatura su scala

Usare QGIS per generare centinaia di mappe

Repository Github:

`github.com/paulbradshaw/QGIS_param`

In 1 ora saprete:

- Come usare la funzionalità **Python di QGIS**
- Come usare la **parametrizzazione** per generare una mappa per ogni area/partner/città
- Consigli per usare **AI** per facilitare la programmazione

Scenario:

La BBC Shared Data Unit sta lavorando a un articolo sulle difese dalle alluvioni. Verrà distribuito alle redazioni di tutto il Regno Unito. Vogliamo generare un'immagine di mappa per la redazione di ogni città, mostrando lo stato delle difese dalle alluvioni nella loro area.

Come creo una
mappa?

Fatelo adesso:
Trovate il file shapefile
Importatelo in QGIS

Seguite la guida nel repository

1. Andate su [AIMS Spatial Flood Defences \(inc. standardised attributes\)](#) e scaricate il file denominato `AIMS_Spatial_Flood_Defences_inc_standardised_attributes.shp.zip`
2. Decomprimete il file
3. Aprite QGIS e create un nuovo progetto (*Project > New*). Salvatelo nella stessa cartella.
4. Selezionate *Layer > Add Layer > Add vector layer*.
5. Nella finestra che appare, cliccate sui ... accanto alla sezione Origine (dove dice *Vector Dataset(s)*) e individuate nella cartella scaricata il file che termina con **.shp**. Cliccate su **Aggiungi**. Dovrebbe apparire sullo sfondo dietro la finestra. Cliccate su **OK** e poi su **Chiudi** per tornare alla mappa.
6. Nell'area *Browser* in alto a sinistra dello schermo, scorrete fino alla sezione XYZ Tiles. Fate doppio clic su OpenStreetMap per aggiungere quel layer di base al vostro progetto (cliccate OK per approvare il cambio allo stesso CRS). Questo dovrebbe ora apparire nell'area Layer.
7. Cliccate e trascinate il layer OSM per spostarlo sotto il layer delle difese dalle alluvioni.

Personalizzate l'aspetto

1. Cliccate con il tasto destro sul layer delle difese dalle alluvioni e selezionate **Proprietà...**
2. Selezionate **Simbologia** per cambiare il colore, l'opacità, la larghezza, lo stile ecc. delle linee/forme

**Come esporto
un'immagine?**

**Fatelo adesso:
Ingrandite un'area
Project > New Print
Layout...**

Create un layout (lo automatizzeremo)

1. Ingrandite la parte della mappa di cui volete creare un'immagine (es. ingrandita su una città)
2. Andate su **Project > New Print Layout...** Dategli un nome (es. cityzoom) e cliccate su **OK**
3. Apparirà la finestra del layout, vuota. Cliccate il pulsante Aggiungi Mappa a sinistra (icona pagina con un più verde)
4. Cliccate e trascinate per disegnare l'area dove volete che appaia la mappa.
5. Andate nella scheda **Proprietà elemento** sul lato destro e scorrete fino a **Posizione e dimensione**. Modificatela con le dimensioni desiderate per l'immagine (cambiate la misura in pixel se è per il web).
6. Passate alla scheda **Layout** accanto, e scorrete fino al pulsante **Ridimensiona** layout. Cliccatelo. L'immagine dovrebbe ora occupare l'intero canvas (potete ingrandire usando i pulsanti di zoom nell'angolo in alto a sinistra)
7. Quando siete soddisfatti dell'immagine andate su **Layout > Export as Image**
8. Chiudete la finestra Print Layout e tornate al progetto QGIS



**Come faccio la
stessa cosa più
volte?**

Se devi fare qualcosa più di una volta...

Plugins > Python Console

Scrivete/aprite Python nella Console **sotto la mappa.**

*Cliccate il pulsante **Show Editor** per aprire/modificare file .py*

PyQGIS library ([documentazione](#))

Fatelo adesso:

Scaricate [il file .py](#)

**Apritelo nella vostra
Console**

Standardise

Python Console

```
1 # Python Console
2 # Use iface to access QGIS API interface or type '?' for more info
3 # Security warning: typing commands from an untrusted source can harm your computer
4
```

>>>



Untitled-0 X

1

Coordinate

357074,195749



Scale

1:152681



Magnifier

100%

Rotation

0.0 °



Render



EPSG:27700



(I passaggi sono descritti nella parte 3 della guida nel repo)

1. Vai al [file raw](#) su GitHub. Fai clic su File > Save As... nel tuo browser e salva il file nella cartella del tuo progetto
2. Rimuovi il commento da questa riga e modifica il percorso in modo che punti alla cartella del tuo progetto

```
# out_prefix = "/Users/paul/Downloads/testqgis/images/"
```

3. Nella console Python di QGIS, apri il file, quindi esegui il codice

Carica gli strumenti che ci permettono di controllare le mappe e i layout di QGIS da questo script

```
from qgis.PyQt.QtCore import QRectF
```

```
from qgis.core import (
```

```
    QgsProject, QgsCoordinateReferenceSystem, QgsPrintLayout, QgsLayoutItemMap,
```

```
    QgsLayoutPoint, QgsLayoutSize, QgsUnitTypes, QgsLayoutItemLabel,
```

```
    QgsLayoutExporter, QgsRectangle, QgsCoordinateTransform, QgsPointXY
```

```
)
```

Carica math per i calcoli numerici, re (regex) per il testo, os per la navigazione file

```
import math, re
```

```
import os
```

Qui viene definita la cartella di output

Usa la libreria os per creare una cartella sul computer in cui salvare le immagini delle mappe esportate.

Verrà salvata nella cartella Download, all'interno di una sottocartella chiamata "qgis_images".

Se la cartella esiste già, non fa nulla – non sovrascrive niente.

```
out_dir = os.path.join(os.path.expanduser("~"), "Downloads", "qgis_images")
```

```
os.makedirs(out_dir, exist_ok=True)
```

```
out_prefix = out_dir + "/"
```

Per sovrascrivere la cartella impostata sopra, decommenta e adatta la riga seguente

```
# out_prefix = "/Users/paul/Downloads/testqgis/images/"
```

#imposta il Sistema di Riferimento delle Coordinate (CRS) - EPSG:4326 viene usato per il GPS
#ma 27700 viene usato per le difese alluvionali nel Regno Unito, quindi usiamo questo per rendere la mappa più rigorosa

```
QgsProject.instance().setCrs(QgsCoordinateReferenceSystem("EPSG:27700"))
```

Ottieni il progetto QGIS attualmente aperto e memorizzalo in una variabile chiamata project

```
project = QgsProject.instance()
```

#Usare EPSG:27700 significa che dobbiamo convertire lat/long in coordinate BNG
prima imposta i sistemi sorgente (EPSG:4326) e destinazione (EPSG:27700)

```
source_crs = QgsCoordinateReferenceSystem("EPSG:4326")
```

```
dest_crs = QgsCoordinateReferenceSystem("EPSG:27700")
```

#poi crea un trasformatore che usa entrambi

```
transformer = QgsCoordinateTransform(  
    source_crs,  
    dest_crs,  
    project  
)
```

```
#crea un nuovo layout di stampa in quel progetto, memorizzalo in una variabile chiamata layout
layout = QgsPrintLayout(project)

#riempilo con le impostazioni predefinite

layout.initializeDefaults()

#assegnagli un nome in QGIS

layout.setName("centres_layout") # Assegna un nome al layout
```

Aggiungi un pannello mappa al layout – questo è il rettangolo che mostrerà effettivamente la mappa.

```
map_item = QgsLayoutItemMap(layout)
```

```
map_item.setFrameEnabled(True) # Disegna un bordo attorno al pannello mappa
```

Posiziona il pannello mappa a 20mm dal bordo sinistro e 20mm dal bordo superiore della pagina.

```
map_item.attemptMove(QgsLayoutPoint(20, 20, QgsUnitTypes.LayoutMillimeters))
```

Imposta il pannello mappa a 257mm di larghezza e 170mm di altezza (circa A4 orizzontale meno i margini).

```
map_item.attemptResize(QgsLayoutSize(257, 170, QgsUnitTypes.LayoutMillimeters))
```

```
layout.addLayoutItem(map_item) # Aggiungi il pannello mappa alla pagina del layout
```

Aggiungi un'etichetta di testo al layout, che mostrerà il nome e le coordinate di ogni località.

```
label = QgsLayoutItemLabel(layout)
```

```
layout.addLayoutItem(label)
```

Posiziona l'etichetta vicino alla parte superiore della pagina (20mm dal sinistro, 5mm dall'alto).

```
label.attemptMove(QgsLayoutPoint(20, 5, QgsUnitTypes.LayoutMillimeters))
```

def crea una funzione (una ricetta) che possiamo usare in seguito. Questa funzione calcola un'area mappa rettangolare centrata su un punto dato. Ha bisogno di quattro ingredienti: x e y del punto centrale; e metà larghezza/altezza (con un valore predefinito per l'ultimo elemento se non fornito).

```
def rect_around(x, y, half_width, half_height=None):
```

```
    # Se non viene fornita un'altezza, usa lo stesso valore della larghezza
```

```
    if half_height is None:
```

```
        half_height = half_width
```

```
    # Quando usata, restituisce un rettangolo che può essere usato per impostare l'area visualizzata dalla mappa.
```

```
    return QgsRectangle(
```

```
        x - half_width,
```

```
        y - half_height,
```

```
        x + half_width,
```

```
        y + half_height )
```

Qui sono memorizzate le **posizioni**

Crea un elenco di luoghi da mappare. Ogni elemento ha una latitudine, una longitudine e un nome.

Partiamo con solo due elementi per i test, ma una volta che funziona, lo espandiamo a tutti i luoghi per cui vogliamo creare immagini

```
listofdicstocsv = [  
    {'lat': 50.844441271809465, 'long': -0.2985307978506628, 'officialname': 'Adur  
District Council'},  
    {'lat': 54.71108952940033, 'long': -3.2472347297148736, 'officialname':  
'Allerdale Borough Council'}  
]
```


Qui viene definita la scala

IMPOSTA SCALA MAPPA: 1 unità sulla mappa corrisponde a 175000 in realtà

MAP_SCALE = 175000

Qui viene definito l'output

```
# Configura lo strumento che salverà il layout come file immagine.
```

```
exporter = QgsLayoutExporter(layout)
```

```
settings = QgsLayoutExporter.ImageExportSettings()
```

```
# IMPOSTA DIMENSIONI IMMAGINE DI OUTPUT
```

```
# 1200x795px per uso web, corrispondente al rapporto del layout A4 nel  
codice precedente
```

```
settings.imageSize = QSize(1200, 795)
```

```
# Scorri ogni posizione nell'elenco e usa la funzione per creare/esportare un'immagine

for rec in listofdicstocsv:

    # Leggi la latitudine, la longitudine e il nome per questa posizione.

    lat = rec.get('lat', float('nan')) # 'nan' usato come segnaposto se il valore manca
    lon = rec.get('long', float('nan'))

    officialname = rec.get('officialname', 'unknown')

    # Se la latitudine o la longitudine mancano, salta questa posizione e continua.

    if math.isnan(lat) or math.isnan(lon):

        print(f"skipping '{officialname}': missing lat/long")

        continue

    # Sposta la vista mappa in modo che sia centrata su questa posizione, zoom usando lon e lat

    transformed_point = transformer.transform( QgsPointXY(lon, lat) )

    # continua...
```

```
# Estrai le coordinate trasformate

x = transformed_point.x()
y = transformed_point.y()

#Crea un rettangolo temporaneo attorno al punto per centrarlo
map_item.setExtent( rect_around(x, y, 1000, 1000) )

#Imposta una scala per controllare lo zoom. Ogni mappa la utilizzerà.
map_item.setScale(MAP_SCALE)

#aggiungi un'etichetta con queste informazioni
label.setText( f"{officialname} | scale 1:{MAP_SCALE:}" )

#aggiorna la mappa prima di esportare
map_item.refresh()

# continua...
```

```
#assegna un nome al file - rimuovi eventuali caratteri problematici
safe_name = re.sub( r'^A-Za-z0-9._-]+', '_', officialname).strip('_')

#usalo per determinare il percorso in cui salvare

out_path = ( f"{out_prefix}" f"{safe_name}.png" )

#ed esporta su quel percorso

result = exporter.exportToImage( out_path, settings )

#se non ha avuto successo

if result != QgsLayoutExporter.Success:

    #mostra un messaggio di errore

    raise RuntimeError(f"Export failed for {out_path}")

#stampa un messaggio alla fine di ogni ciclo

print("wrote", out_path)
```

Come lo faccio
258 volte?

Riepilogo: dove sono memorizzate le posizioni

Crea un elenco di luoghi da mappare. Ogni elemento ha una latitudine, una longitudine e un nome.

Partiamo con solo due elementi per i test, ma una volta che funziona, lo espandiamo a tutti i luoghi per cui vogliamo creare immagini

```
listofdicstocsv = [  
    {'lat': 50.844441271809465, 'long': -0.2985307978506628, 'officialname': 'Adur  
District Council'},  
    {'lat': 54.71108952940033, 'long': -3.2472347297148736, 'officialname':  
'Allerdale Borough Council'}  
]
```

Convertire un CSV in un elenco di dizionari

1. [Scarica il CSV delle città](#) (clicca i tre puntini nell'angolo in alto a destra)
2. **Scrivi questa formula nella cella J2:**

```
=("{ 'lat': "&B2&", 'long': "&C2&", 'officialname':  
' "&A2&" ' }")
```
3. **Copia la formula in basso per ogni riga. Ora hai una colonna di dizionari.**
4. **Usa questa formula per unire i risultati e metti una virgola tra ciascuno**

```
=TEXTJOIN(" , " , TRUE , J:J)
```
5. **Nel tuo codice Python, trova le [parentesi quadre] di apertura e chiusura che contengono il tuo elenco di due elementi, e sostituisci quei due elementi con il nuovo elenco di dizionari, mantenendo le parentesi quadre.**

{lat: 51.5072, 'long': -0.1275, 'officialname': 'London'}, {lat: 52.48, 'long': -1.9025, 'officialname': 'Birmingham'}, {lat: 50.8058, 'long': -1.0872, 'officialname': 'Portsmouth'}, {lat: 50.9025, 'long': -1.4042, 'officialname': 'Southampton'}, {lat: 52.9561, 'long': -1.1512, 'officialname': 'Nottingham'}, {lat: 51.4536, 'long': -2.5975, 'officialname': 'Bristol'}, {lat: 53.479, 'long': -2.2452, 'officialname': 'Manchester'}, {lat: 53.4094, 'long': -2.9785, 'officialname': 'Liverpool'}, {lat: 52.6344, 'long': -1.1319, 'officialname': 'Leicester'}, {lat: 50.8147, 'long': -0.3714, 'officialname': 'Worthing'}, {lat: 52.4081, 'long': -1.5106, 'officialname': 'Coventry'}, {lat: 54.5964, 'long': -5.93, 'officialname': 'Belfast'}, {lat: 53.8, 'long': -1.75, 'officialname': 'Bradford'}, {lat: 52.9247, 'long': -1.478, 'officialname': 'Derby'}, {lat: 50.3714, 'long': -4.1422, 'officialname': 'Plymouth'}, {lat: 51.4947, 'long': -0.1353, 'officialname': 'Westminster'}, {lat: 52.5833, 'long': -2.1333, 'officialname': 'Wolverhampton'}, {lat: 52.2304, 'long': -0.8938, 'officialname': 'Northampton'}, {lat: 52.6286, 'long': 1.2928, 'officialname': 'Norwich'}, {lat: 51.8783, 'long': -0.4147, 'officialname': 'Luton'}, {lat: 52.413, 'long': -1.778, 'officialname': 'Solihull'}, {lat: 51.544, 'long': -0.1027, 'officialname': 'Islington'}, {lat: 57.15, 'long': -2.11, 'officialname': 'Aberdeen'}, {lat: 51.3727, 'long': -0.1099, 'officialname': 'Croydon'}, {lat: 50.72, 'long': -1.88, 'officialname': 'Bournemouth'}, {lat: 51.58, 'long': 0.49, 'officialname': 'Basildon'}, {lat: 51.272, 'long': -0.529, 'officialname': 'Maidstone'}, {lat: 51.5575, 'long': 0.0858, 'officialname': 'Ilford'}, {lat: 53.39, 'long': -2.59, 'officialname': 'Warrington'}, {lat: 51.75, 'long': -1.25, 'officialname': 'Oxford'}, {lat: 51.5836, 'long': -0.3464, 'officialname': 'Harrow'}, {lat: 52.519, 'long': -1.995, 'officialname': 'West Bromwich'}, {lat: 51.8667, 'long': -2.25, 'officialname': 'Gloucester'}, {lat: 53.96, 'long': -1.08, 'officialname': 'York'}, {lat: 53.8142, 'long': -3.0503, 'officialname': 'Blackpool'}, {lat: 53.4083, 'long': -2.1494, 'officialname': 'Stockport'}, {lat: 53.424, 'long': -2.322, 'officialname': 'Sale'}, {lat: 51.5975, 'long': -0.0681, 'officialname': 'Tottenham'}, {lat: 52.2053, 'long': 0.1192, 'officialname': 'Cambridge'}, {lat: 51.5768, 'long': 0.1801, 'officialname': 'Romford'}, {lat: 51.8917, 'long': 0.903, 'officialname': 'Colchester'}, {lat: 51.6287, 'long': -0.7482, 'officialname': 'High Wycombe'}, {lat: 54.9556, 'long': -1.6, 'officialname': 'Gateshead'}, {lat: 51.5084, 'long': 0.1192, 'officialname': 'Slough'}, {lat: 53.748, 'long': -2.482, 'officialname': 'Blackburn'}, {lat: 51.73, 'long': 0.48, 'officialname': 'Chelmsford'}, {lat: 53.61, 'long': -2.16, 'officialname': 'Rochdale'}, {lat: 53.43, 'long': -1.357, 'officialname': 'Rotherham'}, {lat: 51.584, 'long': -0.021, 'officialname': 'Walthamstow'}, {lat: 51.2667, 'long': -1.0876, 'officialname': 'Basingstoke'}, {lat: 53.483, 'long': -2.2931, 'officialname': 'Salford'}, {lat: 51.4668, 'long': -0.375, 'officialname': 'Hounslow'}, {lat: 51.5528, 'long': -0.2979, 'officialname': 'Wembley'}, {lat: 52.1911, 'long': -2.2206, 'officialname': 'Worcester'}, {lat: 51.4928, 'long': -0.2229, 'officialname': 'Hammersmith'}, {lat: 51.5864, 'long': 0.6049, 'officialname': 'Rayleigh'}, {lat: 51.7526, 'long': -0.4692, 'officialname': 'Hemel Hempstead'}, {lat: 51.38, 'long': -2.36, 'officialname': 'Bath'}, {lat: 51.5127, 'long': -0.4211, 'officialname': 'Hayes'}, {lat: 54.527, 'long': -1.5526, 'officialname': 'Darlington'}, {lat: 50.8352, 'long': -0.1758, 'officialname': 'Hove'}, {lat: 50.85, 'long': 0.57, 'officialname': 'Hastings'}, {lat: 51.655, 'long': -0.3957, 'officialname': 'Watford'}, {lat: 51.9017, 'long': -0.2019, 'officialname': 'Stevenage'}, {lat: 54.69, 'long': -1.21, 'officialname': 'Hartlepool'}, {lat: 53.19, 'long': -2.89, 'officialname': 'Chester'}, {lat: 51.4828, 'long': -0.195, 'officialname': 'Fulham'}, {lat: 52.523, 'long': -1.468, 'officialname': 'Nuneaton'}, {lat: 51.5175, 'long': -0.2988, 'officialname': 'Ealing'}, {lat: 51.8168, 'long': -0.8124, 'officialname': 'Aylesbury'}, {lat: 51.6154, 'long': -0.0708, 'officialname': 'Edmonton'}, {lat: 51.755, 'long': -0.336, 'officialname': 'Saint Albans'}, {lat: 53.789, 'long': -2.248, 'officialname': 'Burnley'}, {lat: 53.7167, 'long': -1.6356, 'officialname': 'Batley'}, {lat: 53.5809, 'long': -0.6502, 'officialname': 'Scunthorpe'}, {lat: 52.508, 'long': -2.089, 'officialname': 'Dudley'}, {lat: 51.4575, 'long': -0.1175, 'officialname': 'Brixton'}, {lat: 51.5111, 'long': -0.3756, 'officialname': 'Southall'}, {lat: 55.8456, 'long': -4.4239, 'officialname': 'Paisley'}, {lat: 51.37, 'long': 0.52, 'officialname': 'Chatham'}, {lat: 53.523, 'long': 0.0554, 'officialname': 'East Ham'}, {lat: 51.346, 'long': -2.977, 'officialname': 'Weston-super-Mare'}, {lat: 54.8947, 'long': -2.9364, 'officialname': 'Carlisle'}, {lat: 54.995, 'long': -1.43, 'officialname': 'South Shields'}, {lat: 55.7644, 'long': -4.1769, 'officialname': 'East Kilbride'}, {lat: 52.8019, 'long': -1.6367, 'officialname': 'Burton upon Trent'}, {lat: 53.9919, 'long': -1.5378, 'officialname': 'Harrogate'}, {lat: 53.099, 'long': -2.44, 'officialname': 'Crewe'}, {lat: 52.48, 'long': 1.75, 'officialname': 'Lewes'}, {lat: 52.37, 'long': -1.26, 'officialname': 'Rugby'}, {lat: 51.623, 'long': 0.009, 'officialname': 'Chingford'}, {lat: 51.5404, 'long': -0.4778, 'officialname': 'Uxbridge'}, {lat: 52.58, 'long': -1.98, 'officialname': 'Walsall'}, {lat: 51.475, 'long': 0.33, 'officialname': 'Grays'}, {lat: 51.3868, 'long': -0.4133, 'officialname': 'Walton upon Thames'}, {lat: 51.4002, 'long': -0.1086, 'officialname': 'Thornthorpe Heath'}, {lat: 51.599, 'long': -0.187, 'officialname': 'Finchley'}, {lat: 51.5, 'long': -0.19, 'officialname': 'Kensington'}, {lat: 52.974, 'long': -0.0214, 'officialname': 'Boston'}, {lat: 50.4353, 'long': -3.5625, 'officialname': 'Paignton'}, {lat: 50.88, 'long': -1.03, 'officialname': 'Waterlooville'}, {lat: 53.875, 'long': -1.706, 'officialname': 'Guisely'}, {lat: 51.5565, 'long': 0.2128, 'officialname': 'Hornchurch'}, {lat: 51.4009, 'long': -0.1517, 'officialname': 'Mitcham'}, {lat: 51.4496, 'long': -0.4089, 'officialname': 'Feltham'}, {lat: 52.4575, 'long': -2.1479, 'officialname': 'Stourbridge'}, {lat: 51.375, 'long': 0.5, 'officialname': 'Rochester'}, {lat: 53.691, 'long': -1.633, 'officialname': 'Dewsbury'}, {lat: 51.5135, 'long': -0.2707, 'officialname': 'Acton'}, {lat: 51.449, 'long': -0.337, 'officialname': 'Twickenham'}, {lat: 53.046, 'long': -2.993, 'officialname': 'Wrexham'}, {lat: 53.279, 'long': -2.897, 'officialname': 'Elesmere Port'}, {lat: 54.66, 'long': -5.67, 'officialname': 'Bangor'}, {lat: 51.019, 'long': -3.1, 'officialname': 'Taunton'}, {lat: 52.7225, 'long': -1.2078, 'officialname': 'Loughborough'}, {lat: 51.54, 'long': 0.08, 'officialname': 'Barking'}, {lat: 51.6185, 'long': -0.2729, 'officialname': 'Edgware'}, {lat: 50.8094, 'long': -0.5409, 'officialname': 'Littlehampton'}, {lat: 51.576, 'long': -0.433, 'officialname': 'Ruislip'}, {lat: 51.4279, 'long': -0.1235, 'officialname': 'Streatham'}, {lat: 51.132, 'long': 0.263, 'officialname': 'Royal Tunbridge Wells'}, {lat: 53.35, 'long': -3.003, 'officialname': 'Bebington'}, {lat: 53.25, 'long': -2.13, 'officialname': 'Macclesfield'}, {lat: 52.3028, 'long': -0.6944, 'officialname': 'Wellingborough'}, {lat: 52.3931, 'long': -0.7229, 'officialname': 'Kettering'}, {lat: 51.878, 'long': 0.55, 'officialname': 'Brintree'}, {lat: 52.2919, 'long': -1.5358, 'officialname': 'Royal Leamington Spa'}, {lat: 54.1108, 'long': -3.2261, 'officialname': 'Barrow in Furness'}, {lat: 56.0719, 'long': -3.4393, 'officialname': 'Dunfermline'}, {lat: 53.3838, 'long': -2.3547, 'officialname': 'Altrincham'}, {lat: 54.0489, 'long': -2.8014, 'officialname': 'Lancaster'}, {lat: 53.4872, 'long': -3.0343, 'officialname': 'Crosby'}, {lat: 53.4457, 'long': -2.9891, 'officialname': 'Bootle'}, {lat: 51.5423, 'long': -0.0026, 'officialname': 'Stratford'}, {lat: 51.0792, 'long': 1.1794, 'officialname': 'Fekstone'}, {lat: 55.945, 'long': -3.994, 'officialname': 'Cumbemauld'}, {lat: 51.208, 'long': -1.48, 'officialname': 'Andover'}, {lat: 51.66, 'long': -3.81, 'officialname': 'Neath'}, {lat: 52.488, 'long': -2.05, 'officialname': 'Rowley Regis'}, {lat: 54.2825, 'long': -0.4, 'officialname': 'Scarborough'}, {lat: 55.98, 'long': -3.17, 'officialname': 'Leith'}, {lat: 50.9452, 'long': -2.637, 'officialname': 'Yeovil'}, {lat: 51.451, 'long': 0.052, 'officialname': 'Eltham'}, {lat: 51.5541, 'long': -0.1744, 'officialname': 'Hampstead'}, {lat: 53.125, 'long': -1.261, 'officialname': 'Sutton in Ashfield'}, {lat: 51.4015, 'long': -0.1949, 'officialname': 'Morden'}, {lat: 51.6444, 'long': -0.1997, 'officialname': 'Barnet'}, {lat: 53.4466, 'long': -2.3086, 'officialname': 'Stretford'}, {lat: 51.408, 'long': -0.022, 'officialname': 'Beckenham'}, {lat: 51.5299, 'long': -0.3488, 'officialname': 'Greenford'}, {lat: 51.702, 'long': -0.035, 'officialname': 'Cheshunt'}, {lat: 53.48, 'long': -2.89, 'officialname': 'Kirkby'}, {lat: 51.07, 'long': -1.79, 'officialname': 'Salsbury'}, {lat: 53.4897, 'long': -2.0952, 'officialname': 'Ashton'}, {lat: 51.394, 'long': -0.307, 'officialname': 'Sutton'}, {lat: 53.716, 'long': -1.356, 'officialname': 'Castleford'}, {lat: 51.4452, 'long': -0.0207, 'officialname': 'Catford'}, {lat: 53.3042, 'long': -1.1244, 'officialname': 'Workshop'}, {lat: 53.7492, 'long': -1.6023, 'officialname': 'Morley'}, {lat: 51.743, 'long': -3.378, 'officialname': 'Merthyr Tydfil'}, {lat: 53.555, 'long': -2.187, 'officialname': 'Middleton'}, {lat: 51.2834, 'long': -0.8456, 'officialname': 'Fleet'}, {lat: 50.85, 'long': -1.18, 'officialname': 'Fareham'}, {lat: 53.4487, 'long': -2.3747, 'officialname': 'Urmston'}, {lat: 51.3656, 'long': -0.1963, 'officialname': 'Sutton'}, {lat: 51.578, 'long': -3.218, 'officialname': 'Caerphilly'}, {lat: 51.128, 'long': -2.993, 'officialname': 'Bridgwater'}, {lat: 51.401, 'long': -1.323, 'officialname': 'Newbury'}, {lat: 51.4594, 'long': 0.1097, 'officialname': 'Welling'}, {lat: 51.46, 'long': -2.505, 'officialname': 'Kingswood'}, {lat: 51.886, 'long': -0.521, 'officialname': 'Dunstable'}, {lat: 51.336, 'long': 1.416, 'officialname': 'Ramsgate'}, {lat: 51.393, 'long': 0.478, 'officialname': 'Strood'}, {lat: 53.5533, 'long': -0.0215, 'officialname': 'Cleethorpes'}, {lat: 51.5932, 'long': -0.3894, 'officialname': 'Pinner'}, {lat: 52.606, 'long': 1.729, 'officialname': 'Great Yarmouth'}, {lat: 52.9711, 'long': -1.3092, 'officialname': 'Ilkeston'}, {lat: 53.653, 'long': -2.632, 'officialname': 'Chorley'}, {lat: 51.37, 'long': 1.13, 'officialname': 'Heme Bay'}, {lat: 51.872, 'long': 0.1725, 'officialname': 'Bishops Cleeve'}, {lat: 53.005, 'long': -1.127, 'officialname': 'Arnold'}, {lat: 52.724, 'long': -1.369, 'officialname': 'Coalville'}, {lat: 51.994, 'long': -0.732, 'officialname': 'Bletchley'}, {lat: 51.9165, 'long': -0.6617, 'officialname': 'Leighton Buzzard'}, {lat: 55.86, 'long': -3.98, 'officialname': 'Ardrie'}, {lat: 55.126, 'long': -1.514, 'officialname': 'Blyth'}, {lat: 51.574, 'long': 0.4181, 'officialname': 'Laindon'}, {lat: 51.684, 'long': -4.163, 'officialname': 'Llanelli'}, {lat: 52.927, 'long': -1.215, 'officialname': 'Beeston'}, {lat: 52.4629, 'long': -1.8542, 'officialname': 'South Heath'}, {lat: 55.0456, 'long': -1.4443, 'officialname': 'Whitley Bay'}, {lat: 53.4554, 'long': -2.112, 'officialname': 'Denton'}, {lat: 52.932, 'long': -1.127, 'officialname': 'West Bridgford'}, {lat: 51.6578, 'long': -0.2722, 'officialname': 'Borehamwood'}, {lat: 56.0011, 'long': -3.7835, 'officialname': 'Fakirk'}, {lat: 53.5239, 'long': -2.3991, 'officialname': 'Walden'}, {lat: 51.5878, 'long': -0.3086, 'officialname': 'Kenton'}, {lat: 54.0819, 'long': -0.1923, 'officialname': 'Bridlington'}, {lat: 54.61, 'long': -1.27, 'officialname': 'Billingham'}, {lat: 52.918, 'long': -0.638, 'officialname': 'Grantham'}, {lat: 55.0097, 'long': -1.4448, 'officialname': 'North Shields'}, {lat: 51.947, 'long': -0.283, 'officialname': 'Hitchin'}, {lat: 52.7858, 'long': -0.1529, 'officialname': 'Spalding'}, {lat: 51.36, 'long': 0.61, 'officialname': 'Rainham'}, {lat: 51.978, 'long': -0.23, 'officialname': 'Letchworth'}, {lat: 51.6114, 'long': 0.5207, 'officialname': 'Wickford'}, {lat: 53.4111, 'long': -2.8403, 'officialname': 'Huyton'}, {lat: 51.6717, 'long': -1.2763, 'officialname': 'Abingdon'}, {lat: 51.32, 'long': -2.208, 'officialname': 'Trowbridge'}, {lat: 52.5812, 'long': -1.093, 'officialname': 'Wigston Magna'}, {lat: 51.606, 'long': -1.241, 'officialname': 'Didcot'}, {lat: 51.433, 'long': -0.933, 'officialname': 'Eaerley'}, {lat: 51.459, 'long': 0.138, 'officialname': 'Bexleyheath'}, {lat: 53.4429, 'long': -1.4698, 'officialname': 'Ecclesfield'}, {lat: 53.698, 'long': -2.461, 'officialname': 'Darwen'}, {lat: 53.5333, 'long': -2.2833, 'officialname': 'Prestwich'}, {lat: 51.602, 'long': -3.342, 'officialname': 'Pontypridd'}, {lat: 55.828, 'long': -4.214, 'officialname': 'Rutherglen'}, {lat: 51.1295, 'long': 1.3089, 'officialname': 'Dover'}, {lat: 50.8365, 'long': -0.7792, 'officialname': 'Chichester'}, {lat: 51.2226, 'long': 1.4006, 'officialname': 'Deal'}, {lat: 51.9, 'long': -1.15, 'officialname': 'Bicester'}, {lat: 51.5467, 'long': -0.37, 'officialname': 'Northolt'}, {lat: 55.7742, 'long': -3.9183, 'officialname': 'Wishaw'}, {lat: 51.3652, 'long': -0.1676, 'officialname': 'Carshalton'}, {lat: 53.001, 'long': -1.197, 'officialname': 'Bulwell'}, {lat: 54.591, 'long': -5.68, 'officialname': 'Newtownards'}, {lat: 54.326, 'long': -2.745, 'officialname': 'Kendal'}, {lat: 55.082, 'long': -1.585, 'officialname': 'Cramlington'}, {lat: 52.3353, 'long': -2.0579, 'officialname': 'Bromsgrove'}, {lat: 51.703, 'long': -3.041, 'officialname': 'Pont-y-pŵll'}, {lat: 51.509, 'long': -0.338, 'officialname': 'Hanwell'}, {lat: 51.2279, 'long': -2.3215, 'officialname': 'Frome'}, {lat: 51.5981, 'long': -0.1149, 'officialname': 'Wood Green'}, {lat: 52.5708, 'long': -2.0457, 'officialname': 'Darlaston'}, {lat: 55.181, 'long': -1.568, 'officialname': 'Ashington'}, {lat: 52.9677, 'long': -2.1327, 'officialname': 'Longton'}, {lat: 52.7661, 'long': -0.886, 'officialname': 'Melton Mowbray'}, {lat: 52.606, 'long': -1.9179, 'officialname': 'Aldridge'}, {lat: 53.5452, 'long': -2.3999, 'officialname': 'Farnworth'}, {lat: 51.552, 'long': -0.097, 'officialname': 'Highbury'}, {lat: 53.3761, 'long': -2.1897, 'officialname': 'Chaddle Hulme'}, {lat: 54.62, 'long': -1.58, 'officialname': 'Newton Aycliffe'}, {lat: 52.4299, 'long': -1.9355, 'officialname': 'Bournville'}, {lat: 52.009, 'long': -0.789, 'officialname': 'Shenley Brook End'}, {lat: 54.85, 'long': -1.83, 'officialname': 'Consett'}, {lat: 51.3211, 'long': -0.1386, 'officialname': 'Coulston'}, {lat: 52.566, 'long': -2.0728, 'officialname': 'Bilston'}, {lat: 52.7001, 'long': -2.5157, 'officialname': 'Wellington'}, {lat: 54.663, 'long': -1.676, 'officialname': 'Bishop Auckland'}, {lat: 52.395, 'long': -1.979, 'officialname': 'Longbridge'}, {lat: 52.614, 'long': -2.004, 'officialname': 'Bloxwich'}, {lat: 51.5557, 'long': 0.2512, 'officialname': 'Uxminster'}, {lat: 53.321, 'long': -3.48, 'officialname': 'Rhyll'}, {lat: 52.267, 'long': -2.153, 'officialname': 'Droitwich'}, {lat: 53.5355, 'long': -2.5658, 'officialname': 'Hindley'}, {lat: 53.549, 'long': -2.529, 'officialname': 'Westhoughton'}, {lat: 51.3589, 'long': 1.4394, 'officialname': 'Broadstairs'}

Usa uno slice nel ciclo for per testarlo:

Aggiungere [10:20] dopo il nome dell'elenco significa che ciclerà solo sugli elementi nelle posizioni da 10 a 19

```
for rec in listofdicstocsv[10:20]:
```

```
    # Leggi la latitudine, la longitudine e il nome per questa posizione.
```

```
    lat = float(rec.get('lat', float('nan'))) # 'nan' usato come segnaposto se il valore manca
```

```
    lon = float(rec.get('long', float('nan')))
```

```
    officialname = rec.get('officialname', 'unknown')
```

```
    # Se la latitudine o la longitudine mancano, salta questa posizione e continua.
```

```
    if math.isnan(lat) or math.isnan(lon):
```

```
        print(f"skipping '{officialname}': missing lat/long")
```

```
        continue
```

**Come lo faccio per
aree di dimensioni
diverse?**

**Abbiamo bisogno di
scale diverse:
Grande conurbazione,
area più piccola**

Vibe coding è l'ora?

Usa il meta prompting per chiedere all'AI di progettare il prompt prima

Includi:

- Il contesto (giornalismo)
- L'obiettivo
- Istruzioni dettagliate (nomi specifici, file ecc.)

Sei il mio mentore, un data journalist con oltre un decennio di esperienza nel settore. Hai una **conoscenza statistica avanzata** nonché un sano **scetticismo** quando si tratta sia di dati che di fonti umane.

Ti chiederò aiuto con del codice - sei felice di guidarmi, ma **non vuoi che io perda competenze e diventi troppo dipendente** da te, quindi i tuoi consigli saranno sempre progettati per **costringermi a pensare da solo, imparare nuove competenze e concetti, e praticarli**.

Quando fornisci codice **aggiungi commenti che spiegano ogni passaggio** come se ti rivolgessi a una persona senza esperienza di programmazione. **Io preferisco codice semplice che richiede più righe rispetto a codice complesso in una o due righe.**

Segnala eventuali ipotesi che stai facendo sui dati o sulla mia domanda. Avvisami se la domanda non contiene informazioni sufficienti per rispondere accuratamente, o se il risultato potrebbe essere fuorviante senza contesto aggiuntivo.

Allegato c'è del codice Python che genera immagini di mappe centrate in centinaia di posizioni. **Adattalo in modo che ora generi quelle immagini a due scale diverse.**

Aggiungi commenti al codice per evidenziare la sezione in cui fa questo, in modo che io possa modificarla e provare diversi livelli di zoom.

<https://chatgpt.com/share/6a14152d-42c8-83eb-b7bf-5d8c2470f5d1>

Parametri

[https://github.com/paulbradshaw/QGIS_param/blob/main/qgis_take_pix_t
woSize.py](https://github.com/paulbradshaw/QGIS_param/blob/main/qgis_take_pix_t
woSize.py)

La sezione che imposta i parametri della scala

```
# IMPOSTA SCALE MAPPA:
```

```
MAP_SCALES = [  
    175000,    # scala originale  
    120000    # versione ingrandita  
]
```

La sezione che usa quei parametri

```
# NUOVO: CICLO ATTRAVERSO PIÙ SCALE

for MAP_SCALE in MAP_SCALES:

    #Crea un rettangolo temporaneo attorno al punto per centrarlo
    map_item.setExtent(
        rect_around(x, y, 1000, 1000)
    )

    #Applica la scala corrente
    map_item.setScale(MAP_SCALE)

    #aggiungi un'etichetta con quell'informazione
    label.setText(f"{officialname} | scale 1:{MAP_SCALE:,}")
```

...e aggiungendo al nome del file

#usa questo per determinare il percorso in cui salvare

```
out_path = (
```

```
    f"{out_prefix}"
```

```
    f"{safe_name}_{MAP_SCALE}.png" # NUOVO: AGGIUNGI LA SCALA AL NOME FILE
```

```
)
```

L'amico critico:

Cosa dovrebbe dirti l'AI

Aspetti da considerare

- **Conversione tra il CRS della mappa base e quello dello shapefile**
- **Conversione tra dimensione del layout (mm) e imageSize (px)**
- **Esportazione per la stampa/risoluzione 200dpi**
- **Categorie di scala al posto di numeri grezzi (e più di due)**

Prova questo:
Altri file opzionali nel
repo

Punti chiave

- Se devi farlo più di una volta...
- Il vibe coding è divertente ma il coding con un *mentore* è più gratificante

Risorse aggiuntive:

- https://github.com/paulbradshaw/QGIS_param
- Tim Green: [Il Vibe Coding minaccia il giornalismo: perché le redazioni hanno bisogno di regole ora](#)
- [Newsletter FT](#) sulla metodologia che usa l'AI per assistere nel coding per la mappatura delle aree a rischio alluvione